2.4.4 Explicit and Implicit Methods

All of the methods mentioned so far are said to be *explicit* discretizations, since producing the next approximation point on a trajectory does not require solving any implicit equations defined in terms of the previous one. The Verlet method is not implicit even though $\boldsymbol{Q}$ appears on the right in the second equation, since it is given in an explicit way in terms of $\boldsymbol{q}$ and $\boldsymbol{p}$. Implicitness adds another layer in both analysis and numerical implementation, which, however, in certain applications, is readily justified.

Example 2.6 The *Backward Euler* method,

$$z_{n+1} = z_n + h\boldsymbol{f}(z_{n+1}),$$

is an example of an implicit method. The calculation of a timestep involves solving a system of equations of the form

$$\boldsymbol{g}(\boldsymbol{w}) \equiv \boldsymbol{w} - z_n - h\boldsymbol{f}(\boldsymbol{w}) = 0.$$

The map $\mathscr{G}_t$ is defined implicitly by the equation

$$\mathscr{G}_h(z) = z + h\boldsymbol{f}(\mathscr{G}_h(z)).$$

An implicit method will typically result in a system of nonlinear equations of the form

$$\boldsymbol{g}(z_{n+1}) = \boldsymbol{\tau}_n, \tag{2.25}$$

which will need to be solved at each timestep. The right hand side $\boldsymbol{\tau}_n$ is a vector that depends on the previous time-step z_n, perhaps in a complicated way. We may assume the number of equations represented by (2.25) is equal to the dimension of the space where z is defined, so we have a square nonlinear system. Typically $\boldsymbol{g}$ will depend on the stepsize and coefficients of the method and will have the property that for h sufficiently small, the solution is uniquely defined and is continuously defined in terms of $\boldsymbol{\tau}_n$, that is the mapping $\boldsymbol{g}$ has a bounded and smooth inverse.

Solving the system may then proceed, from an initial guess $z_{n+1}^{(0)}$ by use of Newton's method:

$$z_{n+1}^{(k+1)} = z_{n+1}^{(k)} - \left[\boldsymbol{J}^{(k)}\right]^{-1} (\boldsymbol{g}(z_n^{(k)}) - \boldsymbol{\tau}_n),$$

where $\boldsymbol{J}^{(k)}$ is the Jacobian matrix of the mapping $\boldsymbol{g}$ evaluated at $z_n^{(k)}$, or else an approximation of this Jacobian (assumed to be nonsingular due to the invertibility

of the mapping). The iteration may be recast in the form

$$\boldsymbol{b}_k = -(\boldsymbol{g}(z_n^{(k)}) - \boldsymbol{\tau}_n), \qquad \boldsymbol{J}^{(k)} \Delta z_k = \boldsymbol{b}_k, \qquad z_{n+1}^{(k+1)} = z_{n+1}^{(k)} + \Delta z_k.$$

The potentially demanding steps here are the calculation of the Jacobian matrix and the solution of the linear system.

One way to save computational work is to recycle the Jacobian from a previous timestep. Alternatively, it may be possible to approximate the Jacobian crudely in such a way that the iteration still converges. For example, if the Jacobian matrix is large and can be written in the form

$$\boldsymbol{J}(z) = \boldsymbol{D} + \boldsymbol{E}(z),$$

where $\boldsymbol{E}$ is small in norm and $\boldsymbol{D}$ is a constant sparse matrix, then, in many cases, the Jacobian matrix may be replaced by the constant matrix $\boldsymbol{D}$ and the iteration will still converge. Newton's method (without approximation of the Jacobian matrix) has a remarkable *quadratic* convergence property, meaning that, when the initial guess is close the solution, the errors e_k, e_{k+1} at the kth and $k+1$st iterations satisfy the relation

$$e_{k+1} \leq K e_k^2.$$

This rapid convergence is typically lost when the Jacobian matrix is approximated in some way, and one finds instead

$$e_{k+1} \leq \rho e_k,$$

where $0 < \rho < 1$, i.e., quadratic convergence is replaced by geometric convergence.

In some cases *semi-implicit* methods can be developed which may only require the solution of a low-dimensional nonlinear system at each step. In Chap. 4, we discuss constrained systems for which implicit methods are needed and, in the case of the SHAKE method, for which the nonlinear system that must be solved at each step is of dimension equal to the number of constraints imposed. This is an example of a semi-implicit method.

In other cases, the special structure of the underlying problem may lead to certain efficiencies in the implementation, as when a sparse matrix is obtained in the linear equations that must be solved at each timestep.

Other methods may be substituted for Newton's method for the purpose of solving the nonlinear equations [292] of an implicit method.